FTK Investigation Runbook

Linear disk investigation in FTK Imager + FTK, aligned to the five questions

Run top to bottom. Each step says what to do, what to look for, and what to record. Tool menus shift between FTK 7 and 8; the order holds. Commands and path tables live in the Execution Reference (§3, §7). Record everything in Hunt Notes with the source `FTK` or `FTK Imager`.

Where FTK fits. Memory first (Volatility), live triage second, disk third. Disk work in FTK **proves and dates** what memory and live triage found (Q1–Q3) and answers Q4–Q5. You arrive at step 1 with a lead list: names, paths, IPs, hashes, accounts. FTK searches for them; it does not replace them.

Step 0 — Pre-flight (5 min)

- ☐ FTK database service running; you can log in (FTK won't open a case without it). Credentials come from whoever built the box. Ask, don't guess.
- ☐ Version: `Help → About`. Note it.
- ☐ Target host's **system time zone** known (from live triage, §1.1 / §2.1). You need it in step 3.
- ☐ Output drive/path chosen. Nothing gets written to evidence media.
- ☐ Lead list open in Hunt Notes (IOC tab). These are your search terms in step 6.

Step 1 — Acquire (FTK Imager) (10–20 min)

Skip if an image is provided. Go to step 2.

1. **Memory first if the box is live and not yet captured:** `File → Capture Memory` → output path → `HOST_mem.mem` → Include pagefile (optional) → Capture. Hash it.
2. **Locked hives fast** (if you need the registry before a full image): `File → Obtain Protected Files` → *Password recovery and all registry files*.
3. **Disk:** `File → Create Disk Image` → Physical Drive (whole disk) or Logical Drive (one volume) → destination type **E01** → evidence info → check **Verify images after they are created** → Start.
4. Keep the generated `.txt` log (MD5/SHA-1).

Record: image name, hashes, acquisition time (UTC). This is the integrity reference for every finding.

Step 2 — Verify (2 min)

- Provided image: FTK Imager → `File → Add Evidence Item → Image File` → right-click → **Verify Drive/Image**. Hash must match the provided value or the acquisition log.

- **Record:** "image verified, MD5/SHA-1 match" with the time. If it doesn't match, stop and report. Analysis of an unverified image is not defensible.

Step 3 — Create the case and process (10 min setup, processing runs in background)

1. `File → New Case` → name `CASE_<host>` → case folder on the output drive.
2. Processing profile **Forensics** (not Field Mode, which skips indexing). **Customize:**

Turn on	Leave off (time sinks)
MD5 + SHA-1	OCR
Flag Bad Extensions (file signature analysis)	Explicit Image Detection
Expand Compound Files	Data Carving (add later in step 9 if needed)
dtSearch Text Index	Video thumbnails
KFF, if a library is configured	Cerberus, unless time allows
Registry Reports	

3. Add evidence: **Acquired Image of Drive** → the E01/raw.
4. **Time zone = the target's system time zone.** Wrong zone shifts every timestamp you report.
5. OK → processing starts. **Browse while it runs**: steps 4–5 work on partial results. Index Search (step 6) is incomplete until indexing finishes.

Record: case created, profile, zone set, processing start time.

Step 4 — Fast triage filters (Overview tab) (10 min)

Shrinks the disk to the files that matter. Work in this order:

#	Click	Look for	Feeds
a	Overview → **KFF Alert**	Known-bad hash hits. Fastest win if a library is loaded	Q1
b	Overview → **Bad Extension**	Renamed executables (`.jpg` that is a PE)	Q1
c	Overview → File Category → **Executable**	Sort Created/Modified desc. Anything in `Users\` , `Temp` , `ProgramData` , `Public` , `/tmp` , `/var/www`	Q1, Q5
d	Overview → **Deleted**	Tools removed after use	Q1, Q5
e	Overview → **Encrypted** / high entropy	Packed payloads, staged archives	Q1

Turn on the **KFF Ignorable** filter to hide known-good OS files. Add columns (right-click header): Created, Modified, Accessed, Path, MD5, Bad Extension.

Record: each hit as an entry (time = the file's Created or Modified time in the zone you set, Q1). Bookmark it (step 10 method).

Step 5 — Q1: What is running that should not be? Prove it on disk (15 min)

For each process lead from memory/live triage:

1. **Explore** tab → navigate to its path. Does the file exist? Size, MD5, Created/Modified.
2. Bottom viewer → **Hex**: `MZ` = PE, `7F 45 4C 46` = ELF. Header vs extension.
3. Right-click → **Export** → hash it SHA-256 on the analysis box (`Get-FileHash` / `sha256sum`).
4. Hash-match: filter on that MD5 to find copies elsewhere on the image (staging dirs, other profiles).

Gate G1: unsigned, writable path, or bad parent + file present on disk = acquire and hash. **Record:** file path, hashes, times. Memory process + disk file = **two sources**, Confirmed for Q1.

Step 6 — Index Search your lead list (10 min, once indexing is done)

Index Search tab → one term per line from Hunt Notes IOCs: IPs, domains, filenames, service/task names, account names, webshell strings (`eval(` , `base64_decode`). Use **Live Search** for regex or hex signatures (slower, no index needed).

On each hit: open the containing file, note path + timestamps, bookmark. **Record:** hit location, Q matching what it shows.

Step 7 — Q3: How does it survive a reboot? (15 min)

Explore to the persistence locations (Execution Reference §7B path tables):

Windows	Linux
`Windows\System32\config\SOFTWARE` , `SYSTEM` → right-click → **View in Registry Viewer** → Run/RunOnce, Services (ImagePath, ServiceDll), Winlogon	`/etc/cron*` , `/var/spool/cron/crontabs/`
`Users\<u>\NTUSER.DAT` → Run, RunOnce	`/etc/systemd/system/` , `/lib/systemd/system/` (enabled = symlink in `*.wants/`)
`Windows\System32\Tasks\` (task XML: `<Exec><Command>`)	`/root/` and `/home/*/.bashrc` , `.profile` , `.ssh/authorized_keys`
`ProgramData\...\StartUp\` , `Users\<u>\AppData\Roaming\...\Startup\`	`/etc/ld.so.preload` , `/etc/rc.local`

Windows	Linux
`Windows\System32\wbem\Repository\` (WMI, needs parser)	`/var/www/` (webshell is persistence too)

Gate G3: a persistence entry that points at the Q1 file = **foothold confirmed**. **Record:** entry, its target, the key/file last-write time (Q3).

Step 8 — Q4 and Q5: Who, how in, and when it started (20 min)

Q4 (who / how): export the logs and read them with the Execution Reference queries.

- Windows: `Windows\System32\winevt\Logs\` → export `Security.evtx`, `System.evtx`, `Microsoft-Windows-PowerShell%4Operational.evtx`, Sysmon. On the analysis box: `Get-WinEvent -Path .\Security.evtx -FilterXPath '*[System[(EventID=4624 or EventID=4648 or EventID=4672)]]'`.
- `Users\` → which profile owns the Q1 file? `SAM` in Registry Viewer → accounts, last logon.
- Linux: `/var/log/auth.log*`, `/var/log/apache2/access.log*`, `/etc/passwd` (UID 0), `/home/*/.bash_history`.

Q5 (when): start-time anchors, in priority order:

1. `Windows\Prefetch\<NAME>-<HASH>.pf`: created ≈ first run. Export; times show in File List.
2. `Windows\appcompat\Programs\Amcache.hve` → Registry Viewer → `Root\InventoryApplicationFile`: entry for the file, SHA-1, key last-write.
3. `NTUSER.DAT` → `...\Explorer\UserAssist\{GUID}\Count` (names are ROT13); `SYSTEM` → `...\Services\bam\State\UserSettings\<SID>`.
4. The file's own `$MFT` times last. They are the first thing altered. Check timestomping in Autopsy (File Metadata tab shows `$SI` vs `$FN`) or export `$MFT`.

Gate G4: account + auth method found = scope extends to other hosts. **Gate G5:** first-seen earlier than the alert = intrusion start. **Record:** Q4 and Q5 entries, each citing the artifact.

Step 9 — Optional depth (only if time remains)

- `Evidence → Additional Analysis` → **Data Carving** (executables, archives, scripts only). Carved files have **no path and no reliable timestamps**; corroborate before using.
- Alternate Data Streams appear in the file list; check them on suspicious files.
- `$Recycle.Bin\<SID>\` → `$I` (original path + delete time) / `$R` (content).

Step 10 — Bookmark and report (10 min)

On every hit as you go: right-click → **Create Bookmark** → name it after the finding (`T-04 x.exe in Public`) → comment = why it matters + Q.

Report: `File → Report` → **Bookmarks only**, include hashes and paths → HTML or PDF. Write each finding with the report-out format (Execution Reference §9): finding, host, first/last seen UTC, evidence, **second independent source**, ATT&CK, mitigation.

Pitfalls

- Analyzing before verifying the image hash.
- Wrong time zone on the evidence item. Every reported time is then off by the offset.
- Waiting for processing to finish before looking at anything. Browse Overview while it runs.
- Trusting carved-file timestamps or paths (there are none).
- Expecting FTK to decode Prefetch/Amcache/Shimcache into a timeline. It locates and exports; Registry Viewer, Volatility, or Autopsy interpret.
- Doing memory analysis in FTK. Capture here, analyze in Volatility, add dumped files back as **Individual File** evidence.